

Faculty of Engineering, Department of Computer Science and Engineering

Final Report

Alternating Optimization for LLM Post-Training: Divergence Diagnosis and Verified Solutions

Author: jianghuanyun

Supervisor: Prof. Hoi To Wai

August 2026

Contents

Contents	2
Abstract	3
1 Introduction	4
2 Background	4
2.1 LLM Post-Training	4
2.2 Alternating Least Squares	4
2.3 The 2×2 Factorial Comparison Framework	5
3 The Problem: Systematic Divergence of ALS on Deep Models	5
3.1 The Failure Pattern	5
3.2 Controlled Ablation: Isolating the Cause	6
3.3 Evaluation-Harness Audit	6
4 Diagnosis: Why ALS Diverges on Deep Networks	7
4.1 Residual Amplification	7
4.2 A Positive Feedback Loop	7
5 Attempted Repairs and Their Invalidation	7
5.1 Gradient Injection (A-SYNC): A Timing No-Op	7
5.2 Re-Interpreting the Variant Ranking: A Learning-Rate-Schedule Effect	8
5.3 The Low-Rank Probe: Eliminating Divergence at the Cost of Quality	8
5.4 Soft-Target ALS (Distillation): No Incremental Value	9
6 Verified Solutions	9
6.1 Plain SGD with a Sustained Learning Rate	10
6.2 AdamW Full-Rank Fine-Tuning	10
6.3 AdamW with LoRA	10
7 Conclusion	10

Abstract

Alternating Least Squares (ALS)-based post-training offers a closed-form alternative to gradient-based fine-tuning of large language models (LLMs). However, ALS-based methods that work on shallow models systematically diverge on deep models (≥ 28 layers). This report diagnoses the failure and evaluates verified solutions within a 2×2 factorial comparison framework — optimizer (ASP vs AdamW) $\times$ parameter form (full-rank vs LoRA). A controlled five-condition ablation on Qwen2.5-7B proves that ALS weight modification — not SGD, not perturbation noise, not hook overhead — is the sole sufficient cause of divergence. A causal analysis shows that residual connections amplify ALS perturbations by approximately $1.08\times$ per layer, exceeding SGD's recovery capacity and predicting the observed depth boundary $L \approx 26$. We then test three natural repair strategies — gradient injection of the ALS solution (A-SYNC), low-rank probe heads, and closed-form soft-target (distillation) ALS — and verify through matched-budget controls that none adds value over plain SGD: the injection was a timing no-op in the original implementation, and even corrected, all three paths are statistically indistinguishable from SGD baselines (7B trajectory correlation 0.9998). Within the same framework, three verified solutions stand: plain SGD with a sustained learning rate converges on 28-layer models (6.83 PPL on Qwen2.5-7B), AdamW full-rank fine-tuning achieves the best absolute quality (1.25 PPL), and AdamW with LoRA is the most compute-efficient (10.41 PPL at 0.1% trainable parameters).

1 Introduction

Fine-tuning large language models is the backbone of modern AI deployment, from instruction tuning to domain adaptation. The standard approach — AdamW on all parameters — is extremely compute-hungry: a single 7B model fine-tune requires hundreds of GPU-hours [1, 2]. Post-training methods that reduce this cost without sacrificing quality would make LLM customization dramatically more accessible.

Alternating Least Squares (ALS) offers a fundamentally different approach. Instead of iterating thousands of gradient steps, ALS solves a closed-form least-squares problem on selected layers, $W^* = \operatorname{argmin} \|XW^T - Y\|^2 = (X^T X + \lambda I)^{-1} X^T Y$, where X is the layer's input activations and Y the target. One solve replaces hundreds of gradient steps — the same trick that made ALS the standard for collaborative filtering in the Netflix Prize era [3]. Applied to LLM post-training, however, ALS-based methods break on deep models: every independent attempt on a 28-layer Qwen2.5-7B (11 in total) ended in divergence (NaN within 1–3 ALS cycles), while the same methods converge on shallow models (≤ 24 layers).

This report answers two questions. First, *why* does ALS diverge on deep models? Through a controlled ablation we isolate ALS weight modification as the sole sufficient cause, and a residual-amplification analysis explains the depth boundary. Second, *what works instead*? Within the 2×2 factorial comparison framework, we evaluate three natural repairs — all invalidated by matched-budget controls — and verify three practical solutions: plain SGD with a sustained learning rate, AdamW full-rank fine-tuning, and AdamW with LoRA.

The remainder of this report is organized as follows. Section 2 reviews the background and the 2×2 comparison framework. Section 3 documents the problem — the systematic divergence of ALS on deep models — together with the controlled ablation and an evaluation-harness audit. Section 4 presents the causal theory. Section 5 describes the attempted repairs and their invalidation. Section 6 summarizes the verified solutions, and Section 7 concludes.

2 Background

2.1 LLM Post-Training

Gradient-based fine-tuning is the incumbent method for adapting pre-trained LLMs. AdamW [2] on all parameters delivers the best absolute quality but is compute-expensive; each step requires a full forward and backward pass. LoRA [1] constrains updates to low-rank subspaces ($\Delta W = BA$, $r \ll d$), reducing trainable parameters by 100–1000 $\times$ while retaining most of the quality, and is currently the dominant efficiency method. Both are gradient-based: the optimizer iteratively descends the cross-entropy loss on next-token prediction.

2.2 Alternating Least Squares

ALS solves the linear least-squares problem $W^* = (X^T X + \lambda I)^{-1} X^T Y$ in closed form, replacing hundreds of gradient steps with one matrix solve. In the ASP (Alternating Stochastic optimization with Perturbation) protocol studied here, each cycle comprises an ALS solve on the output projection (lm_head), an SGD phase on the full model, and a perturbation phase that adds noise to escape local structure. The ALS phase directly overwrites layer weights with the closed-form solution — the defining, and as we show, fatal, property.

2.3 The 2×2 Factorial Comparison Framework

To compare optimization algorithms fairly, we adopt a 2×2 factorial design crossing optimizer (ASP vs AdamW) with parameter form (full-rank vs LoRA), giving four protocols: A (ASP, full-rank), B (AdamW, full-rank), C (ASP, LoRA) and D (AdamW, LoRA). Comparisons are FLOPs-normalized: protocols run until a fixed total floating-point-operations budget is consumed, rather than a fixed number of steps, because one ALS solve costs orders of magnitude more than one SGD or AdamW step. All protocols share the same data loader, seeds, hyperparameters, and evaluation protocol. Experiments span OPT-125m (12 layers), Qwen2.5-0.5B (24 layers), and Qwen2.5-7B (28 layers), evaluated on WikiText-2 [4].

3 The Problem: Systematic Divergence of ALS on Deep Models

3.1 The Failure Pattern

Table 1 summarizes the 2×2 matrix at the scale that matters — 800 training steps on Qwen2.5-7B (28 layers), with results replicated over three seeds. The ASP column fails: protocol A (ASP, full-rank) diverges in every attempt, and protocol C (ASP, LoRA) converges but is an order of magnitude worse than AdamW on the same parameter form. The AdamW column is stable at every depth tested, from OPT-125m to Qwen2.5-7B.

Table 1: The 2×2 factorial comparison on Qwen2.5-7B (800 steps, 3 seeds; A also tested on small models).

Protocol	Optimizer	Parameter form	Qwen2.5-7B final PPL	Small models (OPT-125m / Qwen-0.5B)
A	ASP	Full-rank	Diverged (11/11 attempts)	3,000 – 200,000 PPL
B	AdamW	Full-rank	1.25	16.7 – 20.5 / 27.5 – 68 PPL
C	ASP (ALS removed)	LoRA	122 – 142	—
D	AdamW	LoRA	10.41	—

Three observations follow. First, the optimizer main effect is decisive: AdamW dominates ASP under both parameter forms. Second, the interaction is severe — the ASP penalty is catastrophic (divergence) under full-rank but merely large under LoRA, which is why protocol C also excludes ALS (ALS targets full-rank modules and corrupts LoRA adapters). Third, the failure is a property of *depth*, not scale: the same ASP protocol converges on 12- and 24-layer models (though with poor quality) and diverges on the 28-layer model.

3.2 Controlled Ablation: Isolating the Cause

To determine which component of ASP causes divergence, we ran a five-condition controlled experiment on Qwen2.5-7B, holding data, seeds, and budget fixed and varying only which components are active (Table 2).

Table 2: Controlled ablation on Qwen2.5-7B — ALS weight modification is the sole sufficient cause.

#	Condition	ALS	SGD	Perturb	Final PPL	Result
1	SGD-only	✗	✓	✗	53.6	Converges
2	Perturb-only	✗	✗	✓	94.4	Converges
3	ALS(no-op)+SGD	no-op	✓	✗	54.67	Converges
4	ALS-only	✓	✗	✗	$2.0 \times 10^9 \rightarrow 1.1 \times 10^{15}$	Diverges
5	ALS+SGD	✓	✓	✗	$8.3 \times 10^8 \rightarrow 1.4 \times 10^{195}$	Diverges

The verdict is unambiguous: the only conditions that diverge are those containing real ALS weight modification. SGD alone, perturbation noise alone, and even the full ALS machinery with its weight update suppressed (no-op) all converge on the same 28-layer model. ALS-only diverges in a single solve (PPL jumps from 73 to 2.0×10^9). This is the first time the divergence was proven — not assumed — to originate specifically from ALS's weight-modification behavior.

Note on evidence: conditions 3–5 have raw per-step data archived; the final PPL of conditions 1–2 comes from the experiment log (transcribed). Both converge and their values are consistent with condition 3, so the conclusion is unaffected.

3.3 Evaluation-Harness Audit

During verification we discovered two evaluation-harness bugs that had corrupted absolute PPL numbers for the OPT-125m experiments (the 7B numbers, which use the model's own tokenizer, were unaffected). First, the OPT scripts loaded the gpt2 tokenizer; the two vocabularies differ almost entirely (only 2 of the first 50,257 ids map to the same token), so the model was trained and evaluated on a semantically scrambled cross-vocabulary mapping task. Second, padding labels were not masked (labels = input_ids.clone() scores pad positions, which are trivially predictable). With the OPT tokenizer and pad-masked

labels, the pre-trained OPT-125m WikiText-2 test PPL is 73.7 — a sane value — and training becomes a net improvement (73.7 $\rightarrow$ 50.7). All OPT-125m numbers reported below are from the corrected harness; the harness audit itself is documented as a methodological contribution.

4 Diagnosis: Why ALS Diverges on Deep Networks

4.1 Residual Amplification

Why does modifying the output-layer weights destroy a 28-layer model? The answer lies in the residual connection, the architectural feature that lets gradients flow through deep networks. Each transformer layer computes $h_{l+1} = h_l + f_l(h_l)$. When ALS modifies `lm_head`, the perturbation δ propagates backward through every residual connection. Linearizing each layer's response to the perturbation: $\delta_{l+1} = (I + J_l) \cdot \delta_l$, where J_l is the layer Jacobian. The per-layer amplification factor $\|I + J_l\| \approx 1.08$ is a structural property of trained transformers: the identity path preserves perturbations exactly, and each layer's nonlinear response adds ~8% more. After L layers, $\|\delta_L\| = \|\delta_0\| \cdot 1.08^{(L-1)}$.

At 28 layers this gives $1.08^{27} \approx 8\times$ amplification, while SGD's per-cycle recovery capacity is only ~ 0.005 — a 1600:1 asymmetry: the model cannot heal faster than the perturbation grows. The predicted critical depth $L_{\max} = \ln(C_{\text{recovery}}/\|\delta_{\text{ALS}}\|) / \ln 1.08 \approx 26$, which matches the observed boundary (≤ 24 layers converge, ≥ 28 diverge) across all tested architectures.

4.2 A Positive Feedback Loop

A second finding emerged: ALS's perturbation magnitude *increases* across cycles (δ : 0.085 $\rightarrow$ 0.196, $\times 2.3$), because SGD's partial recovery shifts the body's hidden-state distribution, making the next ALS solve find a larger discrepancy. The divergence is therefore self-reinforcing, not self-limiting — a positive feedback loop that explains why damping and clipping patches merely delay the failure rather than prevent it.

5 Attempted Repairs and Their Invalidation

Given the diagnosis, the natural repair is to keep ALS's ability to find good closed-form directions while avoiding the fatal weight modification. We designed and tested three such repairs. All were invalidated by matched-budget controls — a decisive negative result that saves future researchers from plausible but ineffective design patterns.

5.1 Gradient Injection (A-SYNC): A Timing No-Op

A-SYNC converts the ALS solution into a directional gradient bias instead of a weight jump: solve ALS, compute $\delta = W_{\text{ALS}} - W_{\text{before}}$, revert the weights, and inject $\text{sync} \cdot \delta$ into the gradient buffer before the SGD update. Rigorous verification invalidated this

algorithm at two levels. First, the injection as originally implemented was a *timing no-op*: it was added to the gradient buffer *after* `optimizer.step()` and cleared by the next `zero_grad()`, so it never reached any parameter update. Second, even with corrected timing, the injection is redundant with the cross-entropy gradient: a matched-budget pure-SGD control on Qwen2.5-7B (identical 4800 steps, $\text{lr}=2\text{e-}4$, weight decay, evaluation points) reproduces the “A-SYNC” trajectory almost exactly (Table 3).

Table 3: Decisive control — the gradient injection is vacuous on Qwen2.5-7B.

Metric	A-SYNC (96c)	Pure SGD (96c)
Final PPL	6.82	6.83
Trajectory correlation	—	0.99981
Mean per-cycle PPL difference	—	0.116

The two trajectories are statistically indistinguishable. The convergence previously attributed to A-SYNC is the convergence of plain SGD itself. An OPT-125m timing diagnostic confirmed the mechanism: injection timing $A \approx B$ (pure SGD) $\approx C$ (fixed) within noise, and a correctly-timed sync sweep $\{0.05, 0.5, 2.0\}$ never beat pure SGD (550–559 vs 536 PPL).

5.2 Re-Interpreting the Variant Ranking: A Learning-Rate-Schedule Effect

Our earlier 12-variant exploration of A-SYNC appeared to show an algorithm ranking (CONSTANT best, then Cosine, then Vanilla). With the injection proven vacuous, the ranking must be driven by a different variable — and the natural candidate is the learning-rate schedule, which differs across variants. We verified this directly with pure SGD (no ALS machinery at all) on OPT-125m, in the corrected harness (Table 4).

Table 4: lr-schedule verification (pure SGD, OPT-125m, 16 cycles, corrected harness).

Learning-rate schedule	Final PPL
Fixed $\text{lr} = 2\text{e-}4$ (mirrors CONSTANT)	50.7
Cosine decay $\rightarrow 0$ (mirrors Cosine)	54.7
Exp decay $\times 0.8$ per cycle (mirrors Vanilla)	57.1

The ranking fixed < cosine < exp reproduces the 7B variant ranking (CONSTANT 6.82 < Cosine 13.2 < Vanilla 25.8) exactly. A sustained learning rate sustains SGD convergence; a decaying schedule starves the tail of progress. The perceived algorithm ranking was a learning-rate schedule effect, not an injection-strength effect.

5.3 The Low-Rank Probe: Eliminating Divergence at the Cost of Quality

The second repair keeps ALS but confines its closed-form solve to a low-rank probe head added in parallel to `lm_head` (forward: `orig_fwd(x) + probe(x)`), with the probe's output weight owned by ALS and its input weight trained by SGD. This does eliminate divergence — confirming the residual-amplification mechanism, since a low-rank readout cannot move the body — but it caps quality: on Qwen2.5-7B the probe ($r=64$) reaches 22.8 PPL versus 6.83 for pure SGD. A rank sweep on OPT-125m in the corrected harness shows why (Table 5): no rank adds any quality over pure SGD, even with full-batch ALS solves.

Table 5: Probe-rank sweep (OPT-125m, 16 cycles, corrected harness).

Condition	Final PPL
Pure SGD (no probe)	50.7
Probe $r=64$	50.7
Probe $r=256$	50.6
Probe $r=1024$	50.7

The reason is structural: a closed-form solve of a one-hot cross-entropy objective recovers the same descent direction as the CE gradient itself, so the ALS solve is redundant with the body's SGD. The low-rank bottleneck adds a trainable representation (`probe.inp`) whose value SGD can learn directly, without the closed-form solve.

5.4 Soft-Target ALS (Distillation): No Incremental Value

The one genuinely non-redundant ALS objective is distillation: matching a teacher's soft logit distribution, which a one-shot closed-form solve could in principle achieve in a way SGD cannot trivially replicate. We trained a teacher (pure SGD, 16 cycles on OPT-125m) and compared two students on the soft-KL objective: one trained by SGD and one by closed-form soft-target ALS each cycle (A-KD, rank 256). The closed-form solver provided no incremental value: `kd_als` 52.6 vs `kd_sgd` 52.4, within noise. The closed-form ridge solve and SGD descend to the same fixed point, and the marginal gain of the exact solve is unmeasurable.

Together, Sections 5.1–5.4 close the design space of “salvaging ALS”: gradient injection (no-op or redundant), low-rank probing (no quality gain), and soft-target distillation (no incremental value). In every case a matched pure-SGD or SGD-on-the-same-objective control matches or beats the ALS mechanism.

6 Verified Solutions

The 2×2 framework provides the answer to “what works instead”. Table 6 summarizes the three verified solutions and their evidence.

Table 6: Verified solutions within the 2×2 framework.

Solution	Evidence (Qwen2.5-7B unless noted)	Status
Plain SGD, fixed lr	6.83 PPL in 4800 steps (power-law tail, asymptote ≈ 3.5)	Converges on 28 layers
AdamW full-rank	1.25 PPL at 800 steps (N=3)	Best absolute quality
AdamW + LoRA	10.41 PPL at 800 steps (N=3), $\sim 0.1\%$ trainable parameters	Best compute efficiency

6.1 Plain SGD with a Sustained Learning Rate

Plain SGD post-training — no ALS, no perturbation, no injection — converges on the 28-layer Qwen2.5-7B to 6.83 PPL in 4800 steps with a power-law tail (still -0.013 PPL per cycle at cycle 88; fit $\text{PPL}(c) = 3.48 + 61.3 \cdot c^{(-0.70)}$). The critical hyperparameter is a sustained learning rate: decaying schedules plateau (Section 5.2). This is the minimal convergent baseline that the ALS family could not match on deep models.

6.2 AdamW Full-Rank Fine-Tuning

AdamW on all parameters remains the quality champion: 1.25 PPL on Qwen2.5-7B at 800 steps, replicated over three seeds. Its cost is high — this protocol consumes orders of magnitude more FLOPs than the LoRA variants — but it sets the quality ceiling that any alternative must approach.

6.3 AdamW with LoRA

AdamW on LoRA adapters ($r=8$) achieves 10.41 PPL at roughly 0.1% of trainable parameters — the most compute-efficient verified solution, at a modest quality cost versus full-rank AdamW. On shallow models the FLOPs-normalized comparison is even clearer: LoRA is roughly two orders of magnitude more compute-efficient than full-rank training, and AdamW beats the ASP family on every axis.

7 Conclusion

This project began with a persistent failure — ALS-based post-training diverging on deep LLMs — and produced a rigorously established diagnosis, a causal theory, and a hard-won negative result. Within the 2×2 factorial comparison framework we showed that: (1) the problem is ALS weight modification interacting with residual amplification ($\rho \approx 1.08$ per layer, $L_{\text{max}} \approx 26$), proven by a controlled five-condition ablation; (2) all three natural repairs — gradient injection, low-rank probing, and soft-target distillation — are invalidated by matched-budget controls, with the injection itself a timing no-op in the original implementation; and (3) the framework supplies verified solutions: plain SGD with a sustained learning rate converges on 28-layer models, AdamW full-rank gives the best quality, and AdamW with LoRA the best efficiency.

The project's lasting value is methodological: a reproducible diagnostic protocol that distinguishes real algorithmic contributions from no-op components, an honest account of how a timing error can make a null effect masquerade as an algorithmic contribution until it is controlled against, and a corrected evaluation harness with documented audits. For practitioners, the practical answer is simple: on deep models, remove the ALS machinery and let gradient descent do its job.